Explicación de códigos de la Sesión 6: Respaldos y recuperación ante ataques

1. Introducción
Este informe explica la finalidad de los códigos de la guía Sesión 6, centrada en la auditoría de cambios, la creación de copias de seguridad y la recuperación de una base de datos. Los ejemplos utilizan juego_online y combinan sentencias SQL con scripts de Bash ejecutados desde una terminal.

2. Objetivo
Explicar para qué sirve cada bloque de código y sus instrucciones principales, diferenciando las operaciones realizadas en MySQL/phpMyAdmin de las que corresponden a la terminal del sistema operativo.

3. Diferencia entre SQL y Bash
SQL se utiliza para crear tablas, consultar o modificar registros y definir triggers. Bash permite automatizar tareas del sistema, generar archivos de respaldo, organizar carpetas y ejecutar herramientas de MySQL. Los scripts .sh de esta guía no deben pegarse en la pestaña SQL de phpMyAdmin.

4. Código 1: Auditoría mediante TRIGGER
Este informe explica la finalidad de los códigos de la guía Sesión 6, centrada en la auditoría de cambios, la creación de copias de seguridad y la recuperación de una base de datos. Los ejemplos utilizan juego_online y combinan sentencias SQL con scripts de Bash ejecutados desde una terminal.

Explicar para qué sirve cada bloque de código y sus instrucciones principales, diferenciando las operaciones realizadas en MySQL/phpMyAdmin de las que corresponden a la terminal del sistema operativo.

SQL se utiliza para crear tablas, consultar o modificar registros y definir triggers. Bash permite automatizar tareas del sistema, generar archivos de respaldo, organizar carpetas y ejecutar herramientas de MySQL. Los scripts .sh de esta guía no deben pegarse en la pestaña SQL de phpMyAdmin.

monedas de los jugadores: valor anterior, valor nuevo, cuenta de base de datos y fecha del evento

- cuenta de MySQL/MariaDB que ejecutó el cambio.
- **fecha DATETIME DEFAULT CURRENT_TIMESTAMP** Registra automáticamente

Archivo: scripts/01_auditoria_trazas.sql. Registra automáticamente los cambios realizados en las monedas de los jugadores: valor anterior, valor nuevo, cuenta de base de datos y fecha del evento.

- **USE juego_online;** Selecciona la base de datos sobre la que se ejecutará el código.
- **CREATE TABLE IF NOT EXISTS Auditoria_Monedas (...);** Crea la tabla de auditoría solo si todavía no existe.
- **auditoria_id INT AUTO_INCREMENT PRIMARY KEY** Genera un identificador único y automático para cada registro.
- **jugador_id INT NOT NULL** Identifica al jugador afectado y exige que el campo tenga un valor.
- **monedas_antes / monedas_despues** Almacenan la cantidad de monedas anterior y posterior al cambio.
- **operacion VARCHAR(20)** Indica el tipo de operación; en este ejemplo se registra UPDATE.
- **usuario_bd VARCHAR(100)** Guarda la cuenta de MySQL/MariaDB que ejecutó el cambio.
- **fecha DATETIME DEFAULT CURRENT_TIMESTAMP** Registra automáticamente la fecha y hora del evento.
- **DROP TRIGGER IF EXISTS auditoria_monedas;** Elimina el trigger anterior si existe para poder crearlo nuevamente.
- **DELIMITER //** Cambia temporalmente el delimitador para que los puntos y coma internos no finalicen la definición del trigger.
- **CREATE TRIGGER ... AFTER UPDATE ON Jugadores** Crea una acción automática que se ejecuta después de actualizar filas de Jugadores.
- **FOR EACH ROW** Hace que el trigger se ejecute por cada fila afectada.
- **OLD.monedas y NEW.monedas** OLD representa el valor anterior y NEW el valor después de la actualización.
- **IF NOT (OLD.monedas <=> NEW.monedas)** Comprueba si el valor cambió. El operador <=> compara de forma segura incluso ante valores NULL.
- **INSERT INTO Auditoria_Monedas ...** Inserta en la auditoría los valores anterior y nuevo, la operación y el usuario.
- **CURRENT_USER()** Obtiene la cuenta de base de datos asociada a la ejecución.
- **DELIMITER ;** Restablece el delimitador habitual de las sentencias SQL.
- **UPDATE Jugadores SET monedas = monedas + 100 WHERE jugador_id = 1;** Aumenta en 100 las monedas del jugador cuyo identificador es 1; sirve como prueba del trigger.

- **SELECT * FROM Auditoria_Monedas ORDER BY fecha DESC;** Muestra los registros de auditoría, del más reciente al más antiguo.

Resultado esperado: si las monedas cambian, el trigger agrega un registro. Si el valor queda igual, no se agrega una nueva traza.

5. Código 2: Respaldo completo con mysqldump

Archivo: scripts/02_backup_completo.sh. Crea una copia lógica de juego_online en un archivo .sql que posteriormente puede utilizarse para reconstruir la base.

- **#!/bin/bash** Indica que el archivo debe ejecutarse mediante Bash.
- **set -euo pipefail** Detiene el script ante errores, variables no definidas y fallos dentro de tuberías.
- **DB_NAME="juego_online"** Define la base de datos que se respaldará.
- **DB_USER="root"** Define el usuario de conexión.
- **BACKUP_DIR="$(dirname "$0")/../respaldos"** Calcula la ruta de la carpeta de respaldos a partir de la ubicación del script.
- **FECHA="$(date +%Y%m%d_%H%M%S)"** Obtiene fecha y hora para distinguir cada respaldo.
- **ARCHIVO=...** Construye el nombre del archivo .sql, incluyendo fecha y hora.
- **mkdir -p "$BACKUP_DIR"** Crea la carpeta de destino si no existe.
- **echo "Generando respaldo..."** Muestra un mensaje informativo en la terminal.
- **mysqldump -u "$DB_USER" -p --single-transaction --routines --triggers "$DB_NAME" > "$ARCHIVO"** Exporta la base al archivo indicado. -u especifica el usuario; -p solicita la contraseña; --single-transaction busca una copia consistente sin bloquear las tablas InnoDB durante toda la operación; --routines incluye rutinas almacenadas; --triggers incluye triggers; > redirige la salida al archivo.
- **echo "Respaldo completo generado..."** Muestra la ubicación del archivo al finalizar.

Resultado esperado: se genera un archivo .sql dentro de respaldos. La contraseña se solicita de forma interactiva y no debe escribirse directamente en el código.

6. Código 3: Binary log y recuperación punto en el tiempo (PITR)

Archivo: scripts/03_backup_incremental_binlog.sh. Documenta la configuración del binary log y el procedimiento para recuperar la base hasta un momento anterior a un incidente. El binlog registra cambios confirmados y, junto con un respaldo completo, permite recuperar operaciones posteriores al respaldo.

- **BINLOG_ARCHIVE_DIR=...** Define la carpeta de referencia para archivar archivos binary log.
- **mkdir -p "$BINLOG_ARCHIVE_DIR"** Crea esa carpeta si no existe.

- **cat <<'EOF' ... EOF** Muestra un bloque de texto de configuración en la terminal. No aplica automáticamente esa configuración al servidor.
- **[mysqld]** Identifica la sección de configuración del servidor MySQL.
- **log_bin = /var/log/mysql/mysql-bin.log** Activa el registro binary log y señala dónde guardarlo cuando se coloca en el archivo real de configuración.
- **server_id = 1** Asigna un identificador al servidor.
- **binlog_format = ROW** Configura el registro de cambios en formato por filas.
- **expire_logs_days = 7** En esta configuración de referencia, indica una expiración de siete días; la disponibilidad de esta opción depende de la versión del servidor.
- **mysqlbinlog --stop-datetime="..." archivo-binlog | mysql ... juego_online** Lee eventos hasta la fecha/hora indicada y envía las operaciones al cliente mysql para aplicarlas sobre la base restaurada.

Importante: el script es principalmente una guía de configuración. Crear una carpeta e imprimir el bloque heredoc no activa por sí solo el binary log. Se necesita configurar my.cnf o my.ini con permisos de administrador y conservar los archivos binlog.

PITR significa Point-In-Time Recovery o recuperación a un punto en el tiempo. Primero se restaura el respaldo completo y después se reproducen los eventos del binlog hasta el momento deseado, normalmente justo antes del incidente.

7. Respaldos programados, respaldos de objetos y snapshots

7.1. Respaldo programado con cron

- **crontab -e** Abre la configuración de tareas programadas del usuario en sistemas compatibles.
- **0 1 * * * comando** Programa la ejecución todos los días a la 1:00 a. m.; los campos corresponden a minuto, hora, día del mes, mes y día de la semana.
- **>> /var/log/backup_juego.log 2>&1** Añade la salida al registro y redirige también los mensajes de error.
- **crontab -l** Muestra las tareas programadas.

Su finalidad es automatizar las copias periódicas y evitar depender de que una persona recuerde ejecutarlas.

7.2. Respaldo de objetos individuales

- **mysqldump -u root -p juego_online Auditoria_Monedas > respaldo_auditoria.sql** Exporta solo la tabla Auditoria_Monedas.
- **mysqldump -u root -p --no-data --routines --triggers juego_online > respaldo_objetos.sql** Exporta estructura y objetos seleccionados sin exportar los datos de las tablas.
- **mysql -u root -p juego_online < respaldo_auditoria.sql** Importa las sentencias del archivo en juego_online; < redirige el contenido del archivo hacia mysql.

Este método permite restaurar una tabla u objetos específicos sin reemplazar necesariamente toda la base.

7.3. Snapshot o imagen de disco

Un snapshot es una copia del estado de un disco o volumen administrada por la infraestructura. A diferencia del archivo .sql de mysqldump, es una copia a nivel de almacenamiento. Puede ayudar a recuperar rápidamente un volumen o servidor, pero no sustituye los respaldos lógicos verificables.

8. Código 4: Simulacro de desastre y recuperación

Archivo: scripts/04_simulacro_desastre_recuperacion.sh. Comprueba si la base puede reconstruirse desde el respaldo más reciente. Guarda un estado de referencia, elimina la base de prueba, la crea nuevamente, importa el respaldo y compara los resultados.

- **DB_NAME="juego_online" y DB_USER="root"** Definen la base y el usuario utilizados en el ejercicio.
- **ULTIMO_RESPALDO="$(ls -t ... | head -n1)"** Busca respaldos coincidentes y selecciona el más reciente por fecha de modificación.
- **if [[-z "$ULTIMO_RESPALDO"]]; then ... exit 1** Si no se encontró un respaldo, detiene el proceso para no eliminar la base sin una copia disponible.
- **SELECT nombre_usuario, nivel, monedas FROM Jugadores ORDER BY jugador_id;** Consulta los datos de los jugadores que se usarán como referencia antes de la prueba.
- **tee estado_antes.txt** Muestra el resultado y guarda una copia en un archivo de texto.
- **DROP DATABASE juego_online;** Elimina la base completa. Es destructivo y solo debe usarse en un entorno de pruebas con respaldo verificado.
- **CREATE DATABASE ... CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;** Crea de nuevo la base vacía con la configuración de caracteres indicada.
- **mysql ... < "$ULTIMO_RESPALDO"** Importa el archivo SQL para reconstruir tablas, datos y objetos incluidos en el respaldo.
- **tee estado_despues.txt** Guarda el estado consultado después de la restauración.
- **diff -q estado_antes.txt estado_despues.txt** Compara los archivos. Si son idénticos, no se detectan diferencias en los datos consultados.

La alternativa gráfica utiliza Exportar, Eliminar e Importar en phpMyAdmin. La comparación verifica solo las columnas consultadas (nombre_usuario, nivel y monedas); no demuestra por sí sola que cada objeto de la base esté íntegro.

9. JavaScript de la página interactiva

El archivo HTML también contiene JavaScript para controlar la guía en el navegador. Este código cambia los pasos visibles, muestra explicaciones, copia fragmentos y ejecuta simulaciones locales.

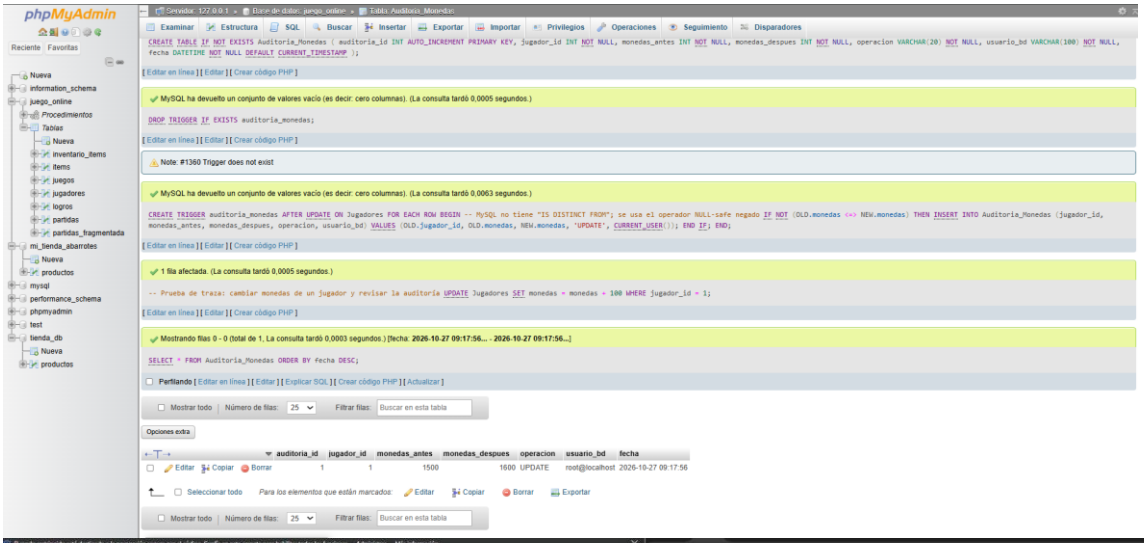
- **document.addEventListener("DOMContentLoaded", ...)** Espera a que el documento esté cargado antes de buscar elementos y asociar eventos.

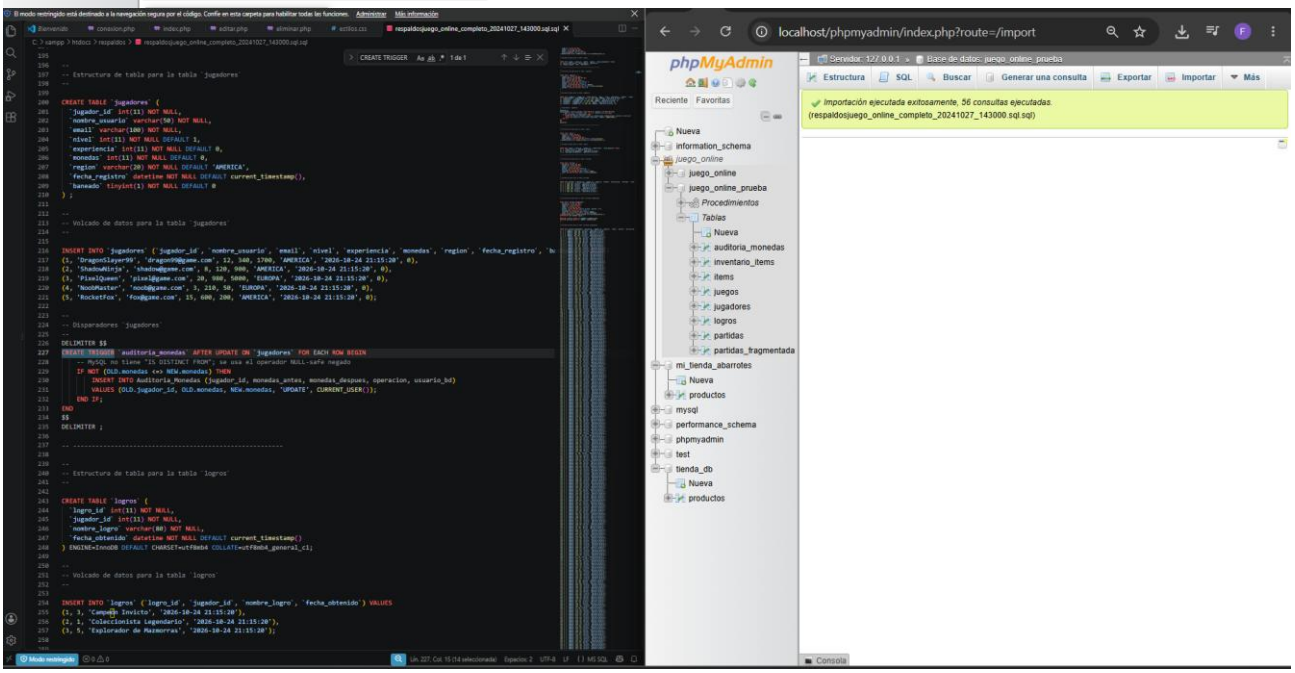
- **querySelectorAll(".step") y querySelectorAll(".panel")** Seleccionan botones de navegación y paneles de contenido.
- **activate(targetId)** Activa el paso elegido, oculta los demás paneles y actualiza el fragmento de la dirección del navegador.
- **addEventListener("click", ...)** Asocia funciones a los clics de botones, como abrir ventanas o cambiar de sección.
- **explainData** Contiene explicaciones línea por línea de los scripts SQL y Bash.
- **navigator.clipboard.writeText(text)** Intenta copiar el código al portapapeles; existe una alternativa si falla.
- **triggerForm y MONEDAS_ACTUALES** Simulan el resultado de un cambio de monedas y muestran si se registraría una traza; no modifican la base real.
- **renderBackupFilename()** Construye un ejemplo de nombre usando fecha y hora del navegador; no genera un archivo .sql.
- **renderPitr()** Actualiza una línea de tiempo simulada para mostrar qué transacciones se recuperarían.
- **runDrill()** Representa en el navegador los pasos del simulacro con datos de ejemplo; no ejecuta DROP DATABASE ni se conecta a MySQL.
- **resetBtn.addEventListener("click", ...)** Limpia los resultados visuales para volver a ejecutar la simulación.

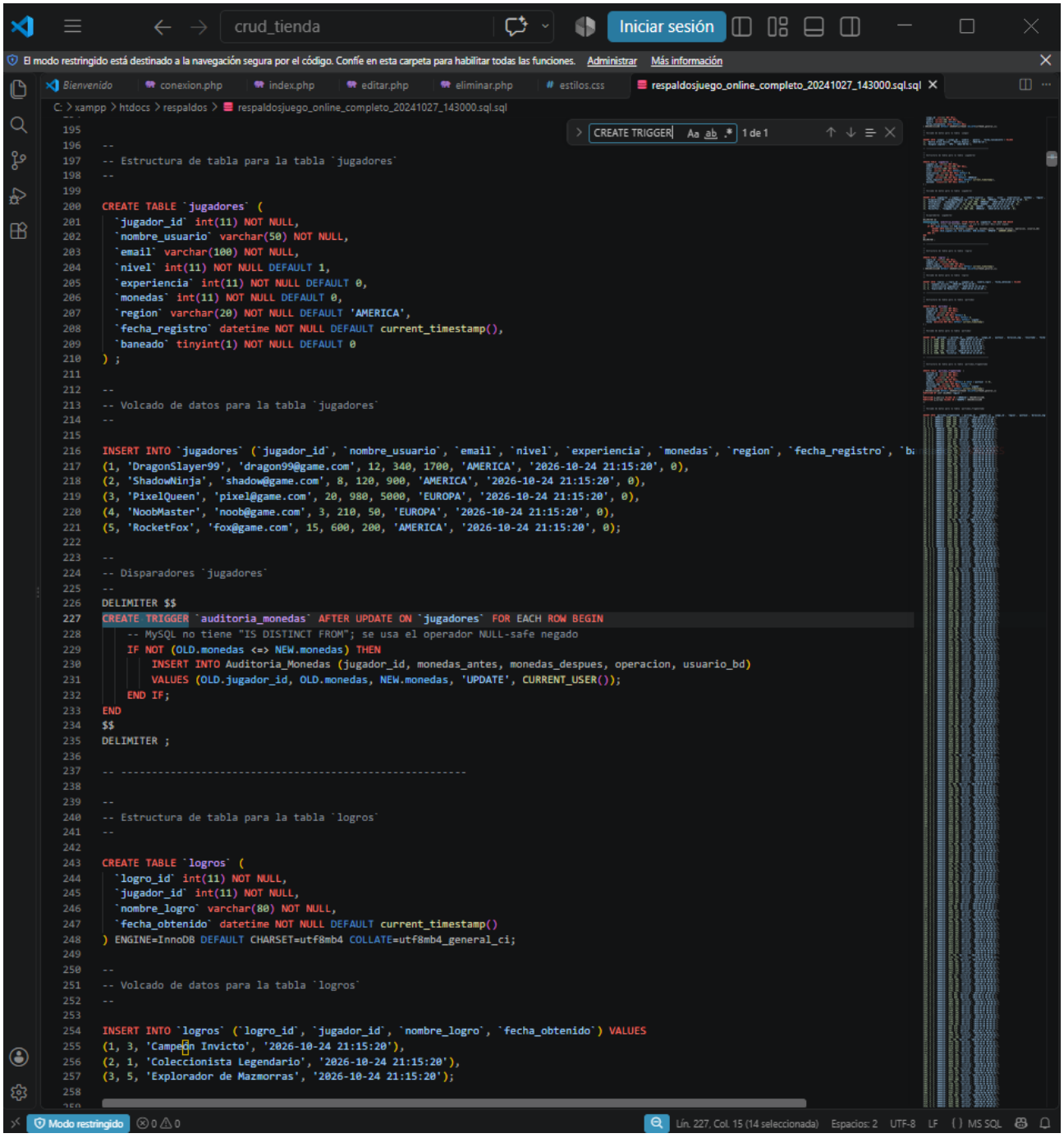
La página tiene dos funciones diferentes: los bloques SQL/Bash describen operaciones que se ejecutarían en el servidor si se ejecutan correctamente; los simuladores JavaScript solo modifican la presentación del navegador.

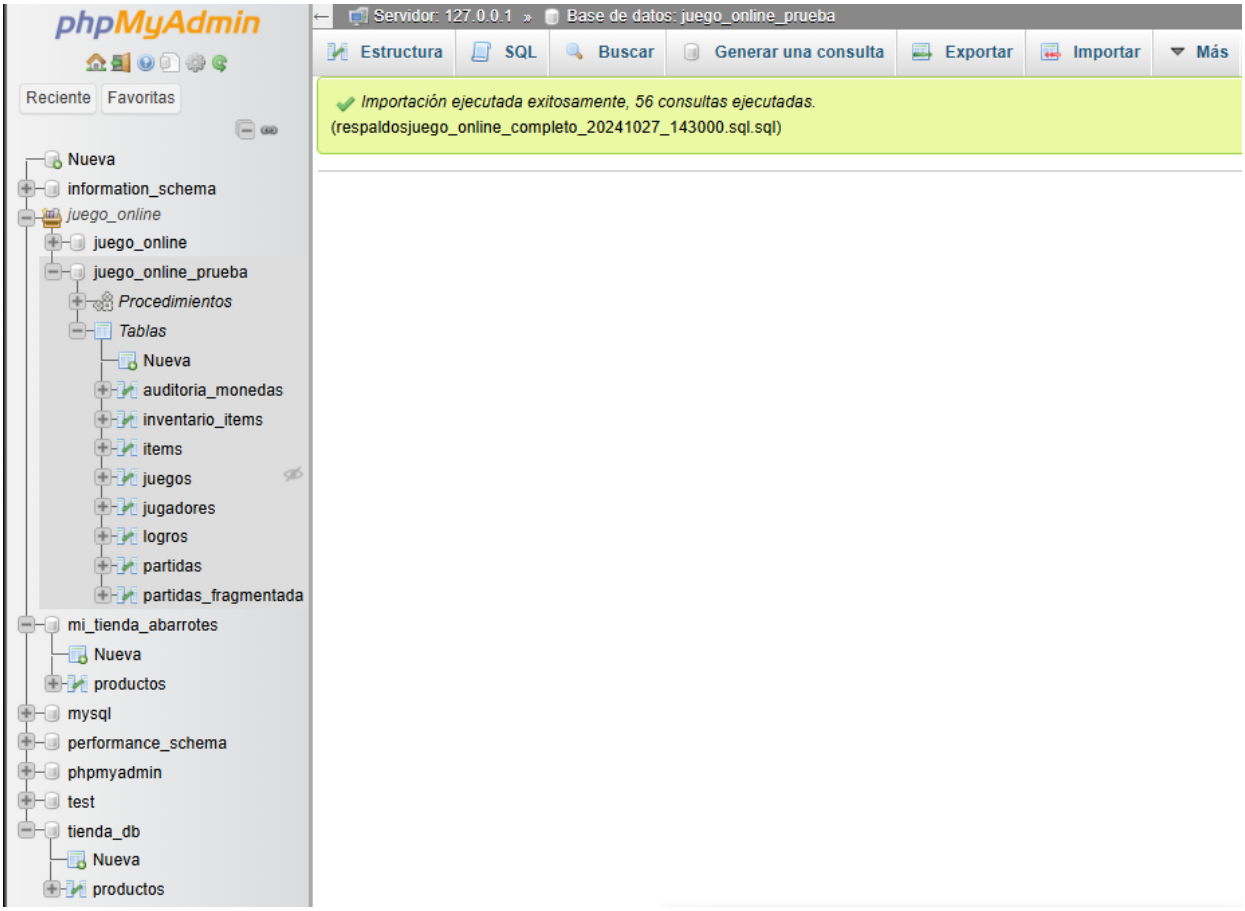
10. Conclusiones

- El trigger registra automáticamente cambios en las monedas y conserva una traza de auditoría.
- mysqldump permite generar respaldos completos o de objetos concretos para restaurar datos y estructuras.
- El binary log, combinado con un respaldo completo, permite recuperar cambios hasta un punto temporal anterior al incidente.
- cron automatiza respaldos periódicos, mientras que los snapshots ofrecen recuperación a nivel de disco.
- El simulacro comprueba el proceso de restauración y compara una selección de datos antes y después.
- La eliminación de una base es destructiva: las pruebas deben realizarse en un entorno local o de laboratorio después de verificar un respaldo válido.


phpMyAdmin
Reciente Favoritas
Nueva
information_schema
juego_online
Procedimientos
Tablas
Nueva
inventario_items
items
juegos
jugadores
logros
partidas
partidas_fragmentada
mi_tienda_abarrotes
Nueva
productos
mysql
performance_schema
phpmyadmin
test
tienda_db
Nueva
productos
Examinar Estructura SQL Buscar Insertar Exportar Importar Privilegios Operaciones Seguimiento Disparadores
CREATE TABLE IF NOT EXISTS Auditoria_Monedas (auditoria_id INT AUTO_INCREMENT PRIMARY KEY, jugador_id INT NOT NULL, monedas_antes INT NOT NULL, monedas_despues INT NOT NULL, operacion VARCHAR(20) NOT NULL, usuario_bd VARCHAR(100) NOT NULL, fecha DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP);
[Editar en línea] [Editar] [Crear código PHP]
MySQL ha devuelto un conjunto de valores vacío (es decir: cero columnas). (La consulta tardó 0,0005 segundos.)
DROP TRIGGER IF EXISTS auditoria_monedas;
[Editar en línea] [Editar] [Crear código PHP]
Note: #1360 Trigger does not exist
MySQL ha devuelto un conjunto de valores vacío (es decir: cero columnas). (La consulta tardó 0,0063 segundos.)
CREATE TRIGGER auditoria_monedas AFTER UPDATE ON Jugadores FOR EACH ROW BEGIN -- MySQL no tiene "IS DISTINCT FROM"; se usa el operador NULL-safe negado IF NOT (OLD.monedas <=> NEW.monedas) THEN INSERT INTO Auditoria_Monedas (jugador_id, monedas_antes, monedas_despues, operacion, usuario_bd) VALUES (OLD.jugador_id, OLD.monedas, NEW.monedas, 'UPDATE', CURRENT_USER()); END IF; END;
[Editar en línea] [Editar] [Crear código PHP]
1 fila afectada. (La consulta tardó 0,0005 segundos.)
-- Prueba de traza: cambiar monedas de un jugador y revisar la auditoría UPDATE Jugadores SET monedas = monedas + 100 WHERE jugador_id = 1;
[Editar en línea] [Editar] [Crear código PHP]
Mostrando filas 0 - 0 (total de 1, La consulta tardó 0,0003 segundos.) [fecha: 2026-10-27 09:17:56... - 2026-10-27 09:17:56...]
SELECT * FROM Auditoria_Monedas ORDER BY fecha DESC;
Perfilando [Editar en línea] [Editar] [Explicar SQL] [Crear código PHP] [Actualizar]
Mostrar todo | Número de filas: 25 Filtrar filas: Buscar en esta tabla
Opciones extra
auditoria_id jugador_id monedas_antes monedas_despues operacion usuario_bd fecha
Editar Copiar Borrar 1 1 1500 1600 UPDATE root@localhost 2026-10-27 09:17:56
Seleccionar todo Para los elementos que están marcados: Editar Copiar Borrar Exportar
Mostrar todo | Número de filas: 25 Filtrar filas: Buscar en esta tabla


CREATE TABLE `jugadores` (
`jugador_id` int(11) NOT NULL,
`nombre_usuario` varchar(50) NOT NULL,
`email` varchar(100) NOT NULL,
`nivel` int(11) NOT NULL DEFAULT 1,
`experiencia` int(11) NOT NULL DEFAULT 0,
`monedas` int(11) NOT NULL DEFAULT 0,
`region` varchar(20) NOT NULL DEFAULT 'AMERICA',
`fecha_registro` datetime NOT NULL DEFAULT current_timestamp(),
`baneado` tinyint(1) NOT NULL DEFAULT 0
) ;
INSERT INTO `jugadores` (`jugador_id`, `nombre_usuario`, `email`, `nivel`, `experiencia`, `monedas`, `region`, `fecha_registro`, `b
(1, 'DragonSlayer99', 'dragon99@game.com', 12, 340, 1700, 'AMERICA', '2026-10-24 21:15:20', 0),
(2, 'ShadowNinja', 'shadow@game.com', 8, 120, 900, 'AMERICA', '2026-10-24 21:15:20', 0),
(3, 'PixelQueen', 'pixel@game.com', 20, 980, 5000, 'EUROPA', '2026-10-24 21:15:20', 0),
(4, 'NoobMaster', 'noob@game.com', 3, 210, 50, 'EUROPA', '2026-10-24 21:15:20', 0),
(5, 'RocketFox', 'fox@game.com', 15, 600, 200, 'AMERICA', '2026-10-24 21:15:20', 0);
DELIMITER $$
CREATE TRIGGER `auditoria_monedas` AFTER UPDATE ON `jugadores` FOR EACH ROW BEGIN
-- MySQL no tiene "IS DISTINCT FROM"; se usa el operador NULL-safe negado
IF NOT (OLD.monedas <=> NEW.monedas) THEN
INSERT INTO Auditoria_Monedas (jugador_id, monedas_antes, monedas_despues, operacion, usuario_bd)
VALUES (OLD.jugador_id, OLD.monedas, NEW.monedas, 'UPDATE', CURRENT_USER());
END IF;
END
$$
DELIMITER ;
CREATE TABLE `logros` (
`logro_id` int(11) NOT NULL,
`jugador_id` int(11) NOT NULL,
`nombre_logro` varchar(80) NOT NULL,
`fecha_obtenido` datetime NOT NULL DEFAULT current_timestamp()
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
INSERT INTO `logros` (`logro_id`, `jugador_id`, `nombre_logro`, `fecha_obtenido`) VALUES
(1, 3, 'Campeón Invicto', '2026-10-24 21:15:20'),
(2, 1, 'Coleccionista Legendario', '2026-10-24 21:15:20'),
(3, 5, 'Explorador de Mazmorras', '2026-10-24 21:15:20');
localhost/phpmyadmin/index.php?route=/import
Importación ejecutada exitosamente, 56 consultas ejecutadas.
(respaldosjuego_online_completo_20241027_143000.sql.sql)
Consola


crud_tienda
Iniciar sesión
El modo restringido está destinado a la navegación segura por el código. Confíe en esta carpeta para habilitar todas las funciones. Administrar Más información
Bienvenido
conexion.php
index.php
editar.php
eliminar.php
estilos.css
respaldosjuego_online_completo_20241027_143000.sql.sql
C: > xampp > htdocs > respaldos > respaldosjuego_online_completo_20241027_143000.sql.sql
CREATE TRIGGER
1 de 1
-- Estructura de tabla para la tabla `jugadores`
CREATE TABLE `jugadores` (
`jugador_id` int(11) NOT NULL,
`nombre_usuario` varchar(50) NOT NULL,
`email` varchar(100) NOT NULL,
`nivel` int(11) NOT NULL DEFAULT 1,
`experiencia` int(11) NOT NULL DEFAULT 0,
`monedas` int(11) NOT NULL DEFAULT 0,
`region` varchar(20) NOT NULL DEFAULT 'AMERICA',
`fecha_registro` datetime NOT NULL DEFAULT current_timestamp(),
`baneado` tinyint(1) NOT NULL DEFAULT 0
) ;
-- Volcado de datos para la tabla `jugadores`
INSERT INTO `jugadores` (`jugador_id`, `nombre_usuario`, `email`, `nivel`, `experiencia`, `monedas`, `region`, `fecha_registro`, `ba
(1, 'DragonSlayer99', 'dragon99@game.com', 12, 340, 1700, 'AMERICA', '2026-10-24 21:15:20', 0),
(2, 'ShadowNinja', 'shadow@game.com', 8, 120, 900, 'AMERICA', '2026-10-24 21:15:20', 0),
(3, 'PixelQueen', 'pixel@game.com', 20, 980, 5000, 'EUROPA', '2026-10-24 21:15:20', 0),
(4, 'NoobMaster', 'noob@game.com', 3, 210, 50, 'EUROPA', '2026-10-24 21:15:20', 0),
(5, 'RocketFox', 'fox@game.com', 15, 600, 200, 'AMERICA', '2026-10-24 21:15:20', 0);
-- Disparadores `jugadores`
DELIMITER $$
CREATE TRIGGER `auditoria_monedas` AFTER UPDATE ON `jugadores` FOR EACH ROW BEGIN
-- MySQL no tiene "IS DISTINCT FROM"; se usa el operador NULL-safe negado
IF NOT (OLD.monedas <=> NEW.monedas) THEN
INSERT INTO Auditoria_Monedas (jugador_id, monedas_antes, monedas_despues, operacion, usuario_bd)
VALUES (OLD.jugador_id, OLD.monedas, NEW.monedas, 'UPDATE', CURRENT_USER());
END IF;
END
$$
DELIMITER ;
-- Estructura de tabla para la tabla `logros`
CREATE TABLE `logros` (
`logro_id` int(11) NOT NULL,
`jugador_id` int(11) NOT NULL,
`nombre_logro` varchar(80) NOT NULL,
`fecha_obtenido` datetime NOT NULL DEFAULT current_timestamp()
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
-- Volcado de datos para la tabla `logros`
INSERT INTO `logros` (`logro_id`, `jugador_id`, `nombre_logro`, `fecha_obtenido`) VALUES
(1, 3, 'Campeón Invicto', '2026-10-24 21:15:20'),
(2, 1, 'Coleccionista Legendario', '2026-10-24 21:15:20'),
(3, 5, 'Explorador de Mazmorras', '2026-10-24 21:15:20');
Modo restringido
Lín. 227, Col. 15 (14 seleccionada)
Espacios: 2
UTF-8
LF
MS SQL



1. OBJETIVO:

Realizar un respaldo completo de la base de datos juego_online incluyendo estructura, datos, procedimientos y disparadores sin interrumpir el servicio.

2. HERRAMIENTAS:

- mysqldump

- phpMyAdmin

- Motor InnoDB

3. METODO 1 - AUTOMATIZADO (CODIGO):

Archivo: 02_backup_completo.sh

Comando:

mysqldump -u root -p --single-transaction --routines --triggers juego_online > ../respaldos/juego_online_completo_FECHA.sql

--single-transaction: Hace respaldo sin bloquear tablas para que los jugadores sigan jugando.

--routines: Incluye procedimientos y funciones.

--triggers: Incluye disparadores.

4. METODO 2 - GRAFICO (PHPMYADMIN):

1. Entrar a phpMyAdmin > Base juego_online > Exportar

2. Metodo: Personalizado > Formato: SQL

3. Seleccionar todo

4. Activar: Agregar sentencia CREATE PROCEDURE / FUNCTION / EVENT

5. Activar: Agregar sentencia CREATE TRIGGER

6. Clic en Exportar

El archivo descargado juego_online.sql es identico al que genera el script.

5. ARCHIVO GENERADO:

/respaldos/juego_online_completo_20241027.sql

Verificacion: El archivo contiene CREATE TABLE, INSERT INTO y CREATE DEFINER=`root`@`localhost` PROCEDURE / TRIGGER, que es la forma en que phpMyAdmin exporta las rutinas.

6. PRUEBA DE RESTAURACION:

Se creo la base juego_online_prueba > Importar > Se selecciono el archivo de respaldo > Importacion exitosa. Esto valida que el respaldo funciona.

7. CONCLUSION:

Se obtuvo un respaldo total por dos vias, por codigo para automatizar y por phpMyAdmin para respaldo manual rapido.